

Activity 04: Exploring System Resources with Python

Sharie Rhea

Southern New Hampshire University

CS 510: Operating Systems Principles and Applications

Brian Enochson

March 1, 2026

The provided Python file, `cpu_memory_details.py`, retrieves and displays information about the central processing unit (CPU) and memory utilization of the system. The script outputs five statistics. First, the current system-wide CPU utilization is shown as a percentage and the number of logical CPU cores is given. For this application, logical cores are defined as “the number of physical cores multiplied by the number of threads that can run on each core” (Rodola, n.d.). The next three statistics are related to system memory. The total physical memory is shown, excluding swap (Rodola, n.d.). Memory in use by all processes is given along with available memory. Available memory is the amount of memory that can be “given instantly to processes without the system going into swap” (Rodola, n.d.).

The above information is retrieved using the cross-platform `psutil` library. The `cpu_percent()` function returns the CPU utilization. Note that calling this function without an interval parameter returns 0.0 on the first call (Rodola, n.d.). To properly retrieve the CPU utilization, either provide a value for the interval argument or wait at least 0.1 seconds between consecutive `cpu_percent()` calls (Rodola, n.d.). The `cpu_count()` function returns the number of logical cores on the system. The `virtual_memory()` function returns many statistics about the system’s memory, each in bytes (Rodola, n.d.). For this application, the total, used, and available statistics are used and converted to gigabytes for legibility. Note that used and free memory may not sum to equal the total memory depending on the platform.

To run the program on your own device, please follow these steps.

1. Install required dependencies.
 - a. Install Python v3.11+.
 - b. Install the `psutil` library: `pip install psutil`

2. Optional: create a virtual environment for pip dependencies before installing.
 - a. `python3 -m venv .venv`
 - b. `source .venv/bin/activate`
3. Run the script in your preferred environment.
 - a. Integrated development environment (IDE): use the “run button” or conventional way to run a file in your preferred IDE.
 - b. Terminal: `python3 cpu_memory_details.py`
4. Observe the formatted output in the terminal or your IDE’s terminal pane. Run multiple times to see the CPU utilization change. Optionally, run an intensive application or use the Linux stress utility for more variation.

There are many real-world motivations to monitor system performance. A high CPU utilization can indicate a system overload. Perhaps too many computationally expensive applications are running (Silberschatz et al., 2023). For servers, this may indicate the need to scale to accommodate more users. If a computer has become unresponsive, which application is causing the behavior can be determined by monitoring each process. A CPU-bound process may “hoard” execution time depending on the scheduler. IO-bound processes may lead to performance issues when memory is slow to access (Silberschatz et al., 2023). In this sense, these statistics can reveal where a process is “bottlenecked”, which can inform future efforts to optimize applications (Silberschatz et al., 2023). For example, if a CPU-bound application is leading to 100% utilization, but only on one core, that process will likely see a major benefit after implementing multithreading. Finally, monitoring system performance is vital for security. Continuously increasing used memory can be a sign of a memory leak. Unusually high CPU utilization may indicate a hidden or malicious process.

An operating system makes many decisions regarding process scheduling and resource allocation. CPU and memory statistics inform these decisions to improve overall performance. The amount of physical memory and number of CPU cores are a hard limitation on how many processes can run simultaneously on a system (Silberschatz et al., 2023). At some point, there simply are not enough resources left to fit in another process. When a process needs to access memory, that page of virtual memory may or may not actually be available in random access memory (RAM). If not, there is a memory fault, similar to a cache miss in a CPU (Silberschatz et al., 2023). If a process constantly switches the pages loaded into RAM—called thrashing—the operating system may deprioritize that process as it will constantly need to wait for memory. Since it's likely that a CPU-bound process does not need to be responsive, these processes are often preempted by the scheduler to allow more interactive applications to run.

I did not experience any major challenges during the creation of this script. The only bug I experienced was an incorrect usage of the `cpu_percent()` function in the `psutil` library. In my first implementation, I called the function with no arguments. I ran the script many times and noticed that the CPU utilization percentage was always 0.0, even when using the `stress` Linux utility. After consulting the documentation, I realized that this is intended behavior, and the function must either be called with an `interval` argument or called more than once with at least 0.1 seconds between calls.

References

- Rodola, G. (n.d.). *psutil documentation*. psutil.readthedocs.io. Retrieved February 25, 2026, from <https://psutil.readthedocs.io/en/latest/>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2023). *Operating System Concepts* (10th ed.). Wiley. <https://learn.zybooks.com/zybook/CS-510-10435.202617-1>